Machine Learning Mid-Term Project

Single-Task and Multi-Task Learning with the "A Century of Portraits" Dataset

---

Objective

In this mid-term project, you will design and implement two models to analyze the "A Century of Portraits" dataset (PyTorch is recommended):

1. A **single-task regression model** that predicts the year a portrait was taken.
2. A **multi-task learning (MTL) network** that jointly predicts the year (regression) and a second task of your choice (gender classification or state classification).

The project is meant to deepen your understanding of regression, classification, multi-task learning, custom dataset creation, and neural network architecture design.

The project is designed to run on **Google Colab's free GPU tier**, and no HPC access is required. *Note: Google Colab is recommended but not required: you are welcome to use other computing resources, including your own GPUs.*

---

1. Dataset

The "A Century of Portraits" dataset contains 37,921 frontal-facing American high school yearbook photos (1905–2013), with metadata such as year, gender, and state.

Download: Dataset (tar.gz, Google Drive) (https://drive.google.com/file/d/1LzSjp1NrXUoN6CQccf5WMGrm9m34tkwo/view)

The archive contains:

- `F/` and `M/` : image folders for female and male portraits
- `train_F.txt` , `train_M.txt` : the training split
- `test_F.txt` , `test_M.txt` : the test split

Each line of a split file gives an image path relative to the dataset root. Gender is given by the folder (`F` / `M`). Year and state are encoded in the image filename (`YEAR_STATE_...`).

More information: A Century of Portraits project page

Requirements

- **Custom PyTorch `Dataset` :** Implement a custom `Dataset` class that loads images and parses metadata

for both models. It must support year regression and your chosen second task.

- **Preprocessing:** Resize images (e.g., 224×224). The images are grayscale; convert them to 3 channels if you use an ImageNet-pretrained backbone.
- **Year normalization (required):** Normalize year targets to [0, 1] using the dataset range (1905–2013) for **both** models, and convert predictions back to years before computing MAE.
- **Missing or incomplete data:** Detect and handle it if any exists, and document what you did.
- **Splits:**
  - Use the **provided train/test split**.
  - Create a **validation set from the training split only** (e.g., 10%), using a seeded split.
  - Use the validation set for all model selection: checkpoints, early stopping, and hyperparameter and loss-weight tuning.
  - **Touch the test set only once**, for the final evaluation. Selecting models or epochs based on test performance will be penalized.

---

2. Getting the Data into Google Colab

Reading tens of thousands of small files directly from a mounted Google Drive is very slow and can time out. Instead, keep the single `.tar.gz` archive on Drive, copy it to Colab's local disk, and unpack it there.

Step 0: Enable the GPU

In Colab, go to **Runtime → Change runtime type → T4 GPU**. Then verify:

```
import torch
print(torch.cuda.is_available(), torch.cuda.get_device_name(0))
```

Step 1: Put the archive on your Google Drive

Open the download link above and either download the `.tar.gz` file and upload it to your Google Drive. Put it in a folder such as `MyDrive/ml_midterm/`.

Step 2: Mount Drive and copy the archive to Colab's local disk

```
from google.colab import drive
drive.mount('/content/drive')
```

```
!mkdir -p /content/data
# Adjust the path and file name to match your Drive
!cp "/content/drive/MyDrive/ml_midterm/<ARCHIVE_NAME>.tar.gz" /content/
```

Step 3: Unpack the archive locally

```
!tar -xzf /content/<ARCHIVE_NAME>.tar.gz -C /content/data
!ls /content/data   # confirm you see F/, M/, and the four split files
```

Point your `Dataset` at `/content/data/...`, **not** at `/content/drive/...`.

Important notes

- Colab's local disk is wiped when the session ends. **Re-run Steps 2–3 at the start of every session.** Copying and unpacking takes only a minute or two.
- Save checkpoints, logs, and plots to **Google Drive** (e.g., `/content/drive/MyDrive/ml_midterm/checkpoints/`) so they survive disconnects.
- Free Colab sessions can disconnect when idle, have a maximum session length, and are subject to usage limits. Implement **checkpoint-and-resume** logic (model, optimizer, scheduler, and epoch) and save a checkpoint after every epoch.
- *Optional speed-up:* Colab has only a couple of CPU cores, so image decoding can bottleneck the GPU. You may preload the images into memory as a NumPy array or tensor once per session.

---

3. Reproducibility: Random Seed

Set all random seeds to `1` at the top of your notebook, and use seeded generators for data splitting and shuffling:

```
import random
import numpy as np
import torch

SEED = 1

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

set_seed()

# Seeded train/validation split
g_split = torch.Generator().manual_seed(SEED)
train_set, val_set = torch.utils.data.random_split(full_train, [n_train, n_val],
generator=g_split)

# Seeded DataLoader shuffling and workers
def seed_worker(worker_id):
    worker_seed = torch.initial_seed() % 2**32
    np.random.seed(worker_seed)
    random.seed(worker_seed)

g_loader = torch.Generator().manual_seed(SEED)
train_loader = torch.utils.data.DataLoader(
    train_set, batch_size=128, shuffle=True,
    num_workers=2, worker_init_fn=seed_worker, generator=g_loader,
)
```

Re-seed (call `set_seed()`) before training **each** model, so the single-task and multi-task runs start from comparable conditions.

---

4. Task Definition

You will train two models:

1. **Single-task model:** Predict the year the portrait was taken as a regression task, outputting a continuous year value (e.g., 1965.7).
2. **Multi-task model:** Jointly predict:
   - **Year** (regression, as above), and
   - **A second task of your choice:**
     - **Gender classification** (binary: male/female), or
     - **State classification** (multi-class: the U.S. state where the portrait was taken, ~27 classes).

Justify your choice of the second task in your report.

---

5. Model Architecture

Design two neural network models in PyTorch:

- **Single-task model:** A CNN for year regression (e.g., ResNet-18/50 or a custom architecture) with a single output neuron and no activation.
- **Multi-task model:** An MTL network with a shared backbone (e.g., ResNet convolutional layers) and task-specific heads:
  - **Year regression head:** a single neuron with continuous output.
  - **Gender/state head:** softmax over 2 outputs (gender) or ~27 outputs (state).
- **Fine-tuning:** You are *encouraged* to fine-tune pre-trained models (e.g., ResNet from `torchvision.models`). Document your strategy, e.g., partial vs. full fine-tuning, or freezing the backbone for the first few epochs.
- **Fair comparison:** Use the **same backbone, input preprocessing, optimizer, learning-rate schedule, and number of epochs** for the single-task and multi-task models, so that any difference in year MAE can be attributed to multi-task learning.

---

6. Implementation

Implement training pipelines for both models in PyTorch, including:

- **Data loading:** Use your custom dataset to load images and metadata.
- **Loss functions:**
  - **Single-task:** L1 (MAE) or MSE loss on normalized years.

    - **Multi-task:** A weighted sum of the losses, e.g., `0.7 × year loss + 0.3 × gender/state loss`, with cross-entropy for classification. Tune the weights on the **validation** set.

- **Optimization:** Use an optimizer (e.g., Adam, AdamW, SGD) with learning-rate scheduling.
- **Training/validation:** Implement loops for both models and monitor performance on the validation set.
- **Evaluation:** Evaluate both models on the test set using:
    - **Year:** MAE (mean absolute error in years)
    - **Gender or state:** accuracy and F1 score (use macro-F1 for state)

  Compute all metrics over the **entire test set at once**, e.g., by collecting all predictions and then calling `sklearn.metrics`. Do **not** average per-batch metrics.
- **Comparison:** Compare single-task and multi-task performance and analyze the impact of MTL. For example, does the second task improve year prediction?

---

7. Deliverables

Code

A well-documented Jupyter notebook (`.ipynb`) that **runs top to bottom in a fresh Colab GPU session without errors**. It must contain:

- The data setup cells (Section 2) and seed setup (Section 3)
- The custom dataset implementation
- The single-task and multi-task model architectures
- Training pipelines and evaluation for both models
- A clearly labeled **final evaluation cell** that prints the test-set results for both models: year MAE for both, plus accuracy and F1 for the second task

Additional rules:

- Submit the notebook **with its outputs saved**.
- The notebook must not depend on files from earlier runs (e.g., silently loading an old checkpoint). If it resumes from a checkpoint, the checkpoint must have been produced by this same notebook, and the logic must be clearly documented.
- The code shown must be the code that produced the outputs shown.

Report

A PDF report (2–4 pages) that includes:

- **Introduction:** Overview of the tasks and their relevance, and justification for your choice of second task.
- **Dataset:** Description of your custom dataset, preprocessing, and train/validation/test setup.
- **Models:** Explanation of the architectures, fine-tuning strategy, and MTL design (e.g., loss weighting).

- **Experiments:** Training details, hyperparameters, and results. Include plots and tables: year MAE, second-task classification metrics, and training/validation curves.
- **Comparison:** Analysis of single-task vs. multi-task performance (e.g., MAE improvement).
- **Discussion:** Challenges, insights, and potential improvements.

---

8. Evaluation Criteria

Component	Weight
Correctness: custom dataset (10%), single-task model (10%), multi-task model (15%)	35%
Creativity: thoughtful task selection and architecture design	5%
Performance: test-set year MAE (see scale below)	20%
Comparison: clear, fair analysis of single-task vs. multi-task models	10%
Documentation: clarity and completeness of code comments and report	10%
Reproducibility: notebook runs without errors and reproduces reported results	20%
Total	**100%**

Performance scale (20%)

Performance is scored on the **test-set year MAE of your better model** (single-task or multi-task), as printed in your final evaluation cell.

- **MAE ≤ 4.0 years:** full 20%
- **Each additional 0.5 years (or part thereof) above 4.0:** –1%
- **Minimum:** 0%

Test MAE (years)	Performance score
≤ 4.0	20%
4.0 – 4.5	19%
4.5 – 5.0	18%
5.0 – 5.5	17%
5.5 – 6.0	16%
…	…
> 13.5	0%

Each range includes its upper bound (e.g., 4.5 counts as 19%). Formula: `score = max(0, 20 - ceil((MAE - 4.0) / 0.5))` for MAE > 4.0.

Second-task expectations: Your multi-task model should reach at least **97.5% test accuracy for gender** or

20% test accuracy for state.

Reproducibility

The grader will re-run your notebook in a fresh Colab GPU session with seed 1. GPU training is not perfectly deterministic even with fixed seeds, so reproduced results within **±0.3 years MAE** (and ±1% accuracy) of your reported numbers are considered reproduced.

---

9. Resources

- **Dataset:** Google Drive download link
- **Dataset background:** A Century of Portraits project page
- **PyTorch documentation:** pytorch.org, for datasets, model building, and training
- **Pre-trained models:** available via `torchvision.models`
- **Compute:** Google Colab with the free T4 GPU runtime

---

10. Submission

Submit via Blackboard:

- The `.ipynb` notebook with outputs saved
- The PDF report

---

11. Tips

Start early

Free Colab GPU time is limited and shared across all your courses. It gets harder to obtain near deadlines, and sessions can disconnect.

Time estimates (total: roughly 10–25 hours)

- Colab setup and custom dataset implementation: 1–4 hours
- Model implementation: 2–6 hours
- Training and evaluation: 6–12 hours
  - Training both models: 2–5 hours of GPU time
  - Debugging training issues: 2–5 hours
  - Evaluation and visualization: 1–2 hours
- Report writing: 1–2 hours

Working efficiently on Colab

- Use mixed-precision training (`torch.autocast` with `torch.amp.GradScaler`) and batch sizes of 64–256 to use the T4 efficiently.
- A pre-trained ResNet-18 typically needs only 10–20 epochs of fine-tuning. ResNet-50 is roughly 3× slower per epoch.
- Keep data on local disk, not Drive (see Section 2).

Getting to MAE ≤ 4

A plain fine-tuned ResNet-18 typically reaches about 5 years MAE on this test set. Getting to 4 or below usually takes some care, for example:

- Matching the loss to the metric (L1)
- A sensible learning rate and schedule, avoiding decay that is too aggressive
- Light data augmentation (e.g., horizontal flips, small crops or shifts)
- Longer training or a larger backbone
- Tuning the MTL loss weights on the validation set

MTL

Tune the loss weights, e.g., with a grid search over the auxiliary weight in [0.1, 0.9] on the validation set. Visualize the loss curves and validation MAE trends for both models.

Expected behavior

- Gender is the easier second task (typically 95–98% accuracy) and tends to pair better with year prediction.
- State is much harder (typically 10–25% accuracy) because the metadata is noisy.
- MTL may improve year MAE by roughly 0.5–1 year, but this is not guaranteed. Report what you actually observe.

Good luck, and enjoy exploring regression and multi-task learning with the "A Century of Portraits" dataset!